Solution: Find Non-Duplicate Number Instances

Problem Statement

Given an array of sorted numbers, move all non-duplicate number instances at the beginning of the array in-place. The relative order of the elements should be kept the same and you should **not use any extra space** so that the solution has constant space complexity i.e., $O(1)$.

Move all the unique number instances at the beginning of the array and after moving return the length of the subarray that has no duplicate in it.

Example 1:

```
Input: [2, 3, 3, 3, 6, 9, 9]
Output: 4
Explanation: The first four elements after moving element will be [2, 3,
6, 9].
```

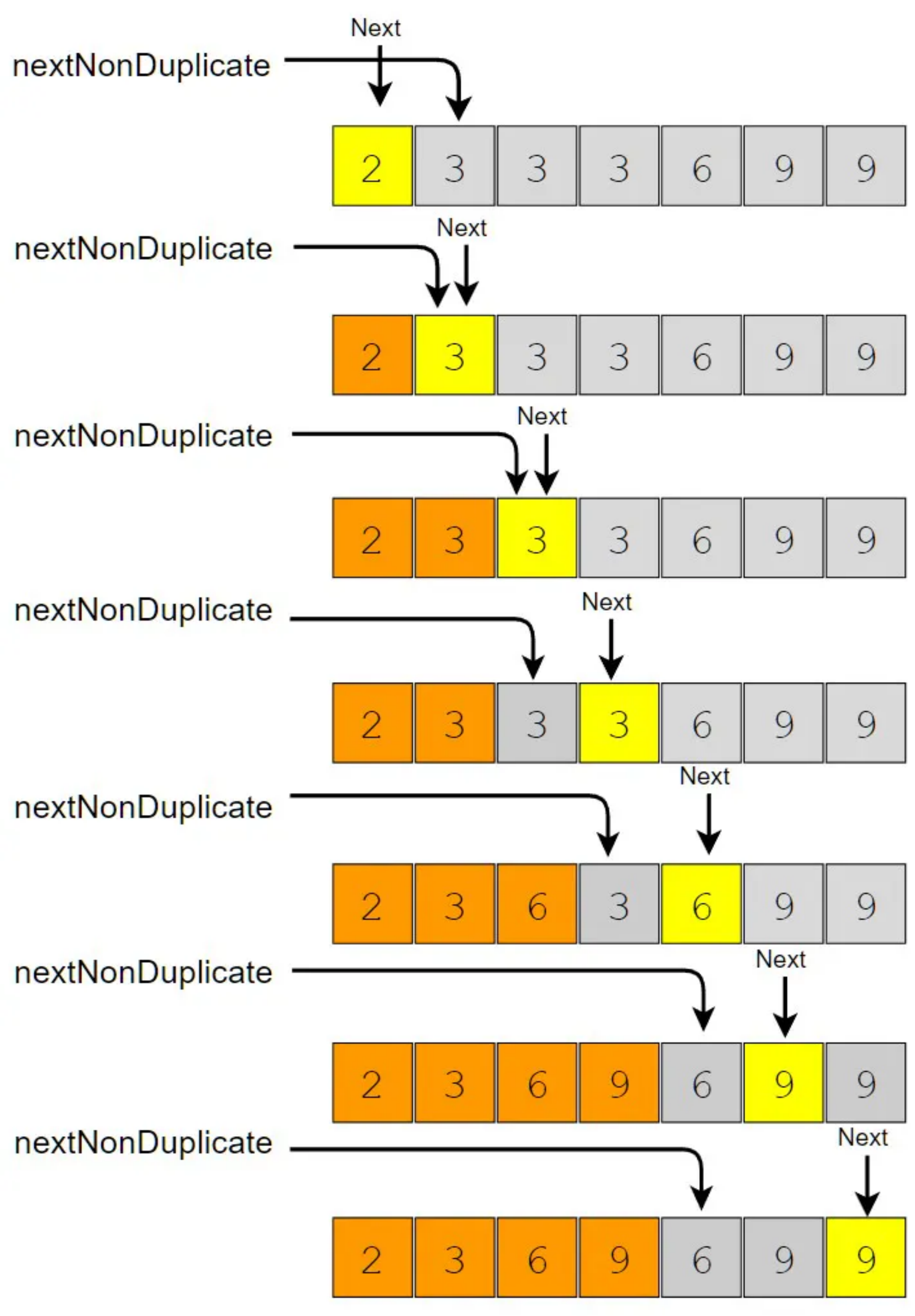
Example 2:

```
Input: [2, 2, 2, 11]
Output: 2
Explanation: The first two elements after moving elements will be [2,
11].
```

Solution

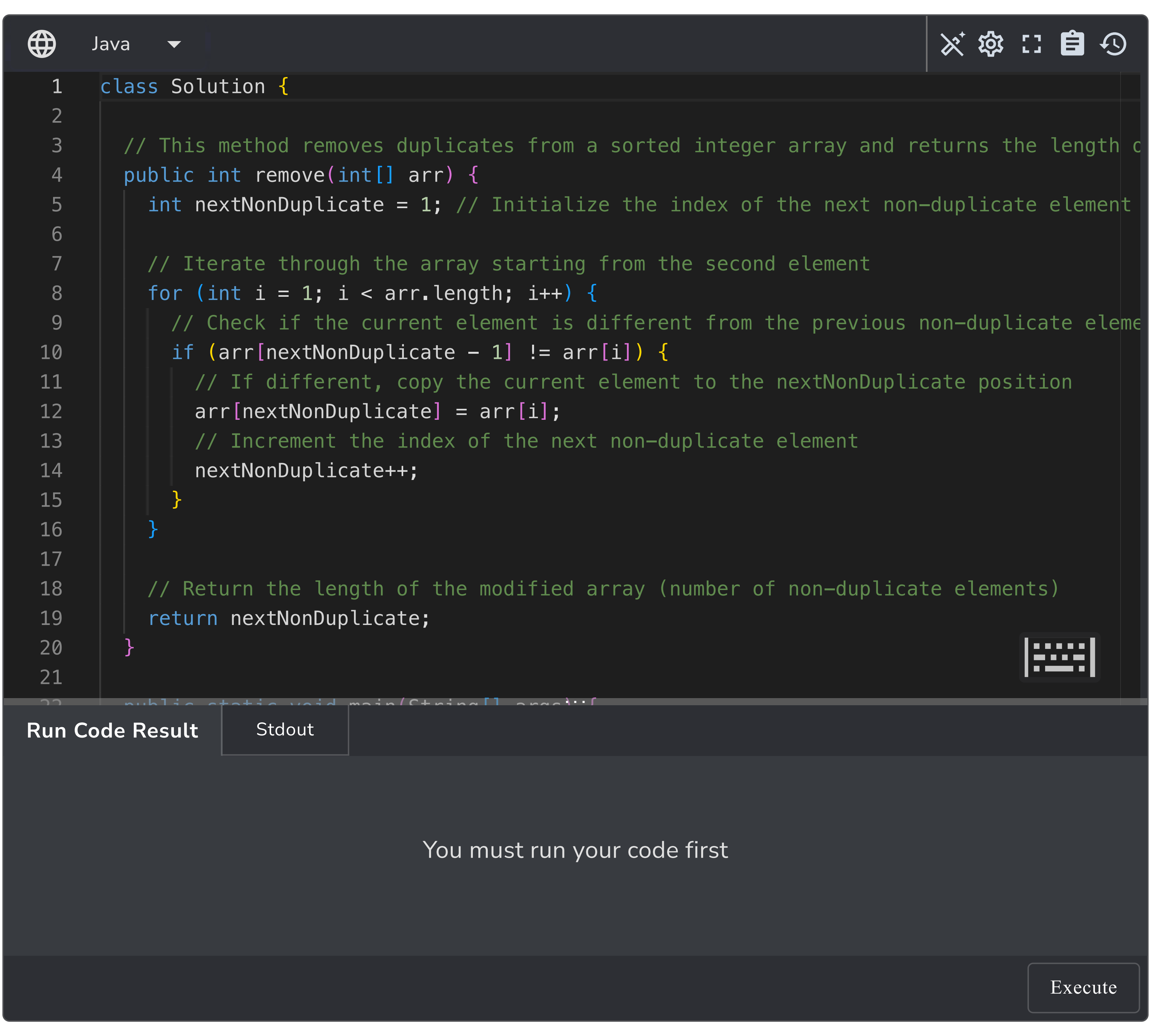
In this problem, we need to separate the duplicates in-place such that the resultant length of the array remains sorted. As the input array is sorted, therefore, one way to do this is to shift the elements left whenever we encounter duplicates. In other words, we will keep one pointer for iterating the array and one pointer for placing the next non-duplicate number. So our algorithm will be to iterate the array and whenever we see a non-duplicate number we move it next to the last non-duplicate number we've seen.

Here is the visual representation of this algorithm for Example-1:



Code

Here is what our algorithm will look like:



```java
class Solution {

  // This method removes duplicates from a sorted integer array and returns the length c
  public int remove(int[] arr) {
    int nextNonDuplicate = 1; // Initialize the index of the next non-duplicate element

    // Iterate through the array starting from the second element
    for (int i = 1; i < arr.length; i++) {
      // Check if the current element is different from the previous non-duplicate eleme
      if (arr[nextNonDuplicate - 1] != arr[i]) {
        // If different, copy the current element to the nextNonDuplicate position
        arr[nextNonDuplicate] = arr[i];
        // Increment the index of the next non-duplicate element
        nextNonDuplicate++;
      }
    }

    // Return the length of the modified array (number of non-duplicate elements)
    return nextNonDuplicate;
  }

```

You must run your code first

Time Complexity

The time complexity of the above algorithm will be $O(N)$, where 'N' is the total number of elements in the given array.

Space Complexity

The algorithm runs in constant space $O(1)$.

Similar Questions

Problem 1: Given an unsorted array of numbers and a target 'key', remove all instances of 'key' in-place and return the new length of the array.

Example 1:

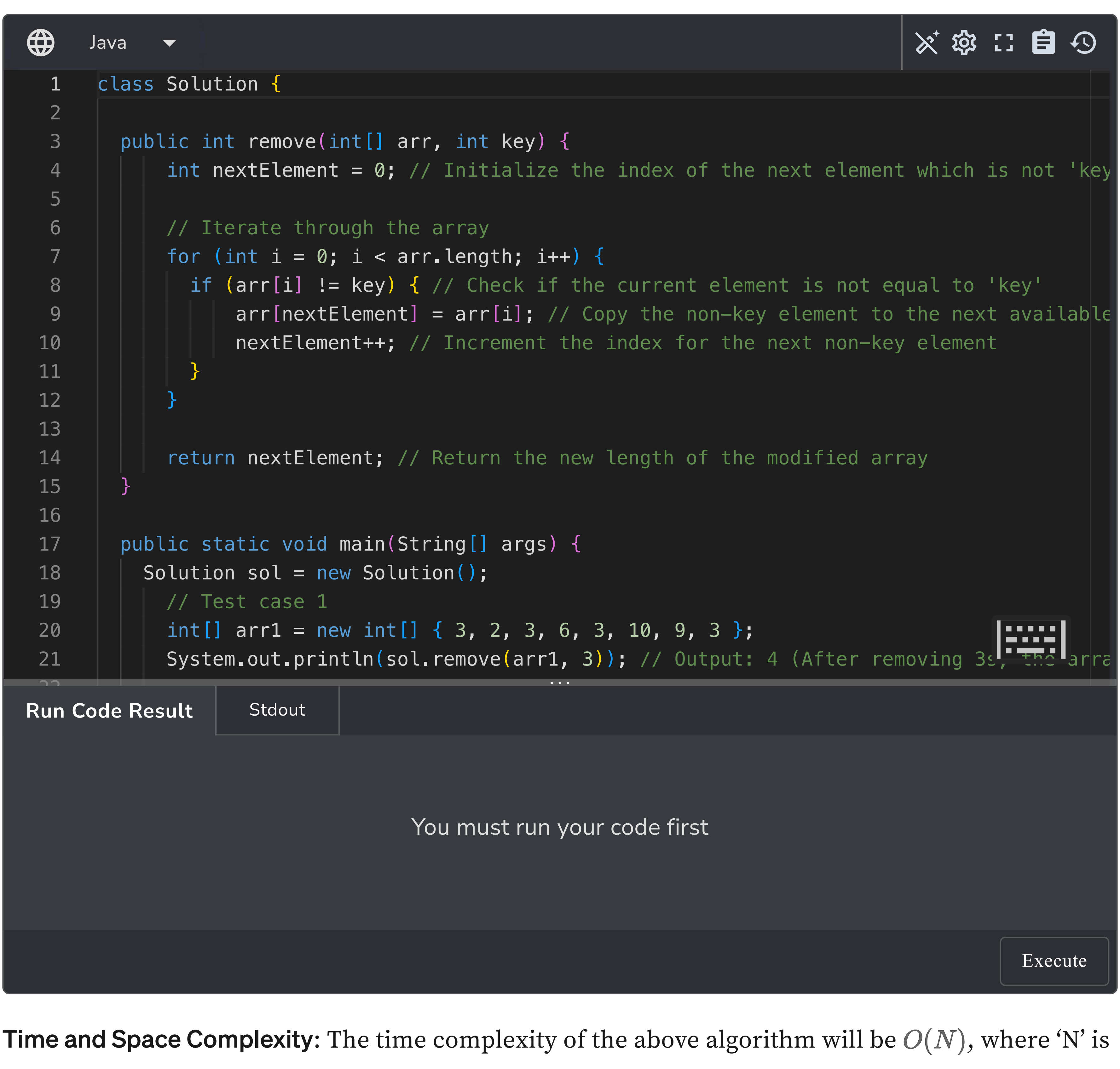
```
Input: [3, 2, 3, 6, 3, 10, 9, 3], Key=3
Output: 4
Explanation: The first four elements after removing every 'Key' will be
[2, 6, 10, 9].
```

Example 2:

```
Input: [2, 11, 2, 2, 1], Key=2
Output: 2
Explanation: The first two elements after removing every 'Key' will be
[11, 1].
```

Solution:

This problem is quite similar to our parent problem. We can follow a two-pointer approach and shift numbers left upon encountering the 'key'. Here is what the code will look like:



```java
class Solution {

  public int remove(int[] arr, int key) {
    int nextElement = 0; // Initialize the index of the next element which is not 'key

    // Iterate through the array
    for (int i = 0; i < arr.length; i++) {
      if (arr[i] != key) { // Check if the current element is not equal to 'key'
        arr[nextElement] = arr[i]; // Copy the non-key element to the next available
        nextElement++; // Increment the index for the next non-key element
      }
    }

    return nextElement; // Return the new length of the modified array
  }

  public static void main(String[] args) {
    Solution sol = new Solution();
    // Test case 1
    int[] arr1 = new int[] { 3, 2, 3, 6, 3, 10, 9, 3 };
    System.out.println(sol.remove(arr1, 3)); // Output: 4 (After removing 3s, the arra
```

You must run your code first

Time and Space Complexity: The time complexity of the above algorithm will be $O(N)$, where 'N' is the total number of elements in the given array.

The algorithm runs in constant space $O(1)$.

← Previous
Find Non-Duplicate Number Instances (Easy)
Next →
Squaring A Sorted Array (Easy)
☐ Mark as Completed